

4x4 Systolic Array AI Accelerator: Complete RTL-to-GDSII Flow Report

Project Report

Hardware Implementation using SkyWater 130nm PDK

Ishaan Singhal
Delhi Technological University

Contents

Abstract	4
1 Introduction	5
1.1 Motivation	5
1.2 Systolic Array Architecture	5
1.3 Project Objectives	5
2 Design Specification and Architecture	6
2.1 System Overview	6
2.2 Design Parameters	8
2.3 Architecture Details	8
2.3.1 Processing Element (PE)	8
2.3.2 Systolic Dataflow with Skewing	8
2.3.3 AXI-Stream Interface	9
3 RTL Design and Implementation	10
3.1 Design Hierarchy	10
3.2 Module Descriptions	10
3.2.1 Processing Element (pe.v)	10
3.2.2 Systolic Array (systolic_array.v)	12
3.2.3 AXI-Stream Buffer (buffer.v)	13
3.2.4 Top-Level Integration (top.v)	14
3.3 Control Unit	16
3.4 Verification and Simulation	17
3.4.1 Testbench Architecture	17
3.4.2 Simulation Results	20
4 Logic Synthesis	21
4.1 Synthesis Flow and Constraints	21
4.1.1 Synthesis Tool and Settings	21
4.1.2 Design Constraints	21
4.2 Synthesis Results	23
4.2.1 Gate-Level Statistics	23
4.2.2 Gate-Level Schematic	23
5 Physical Design Implementation	24
5.1 Design Flow Overview	24
5.2 Floorplanning	25
5.2.1 Objectives	25
5.2.2 Floorplan Specifications	25
5.2.3 Power Distribution Network (PDN)	25
5.3 Standard Cell Placement	26
5.3.1 Placement Strategy	26
5.4 Clock Tree Synthesis (CTS)	28
5.4.1 Objectives	28
5.4.2 CTS Results	28
5.5 Routing	30

5.5.1	Routing Objectives	30
5.5.2	Routing Flow	30
6	Sign-off and Physical Verification	32
6.1	Parasitic Extraction	32
6.2	Final Static Timing Analysis (STA)	32
6.2.1	Setup Timing Analysis (Max Delay)	32
6.2.2	Hold Timing Analysis (Min Delay)	34
6.2.3	Timing Summary	34
6.3	Final Power Analysis	36
6.4	Design Rule Check (DRC)	37
6.5	Layout vs Schematic (LVS)	37
6.6	Final GDSII Layout	38
7	Results and Analysis	39
7.1	Implementation Summary	39
7.2	Performance Analysis	40
7.2.1	Computational Throughput	40
7.2.2	Power Efficiency	40
7.2.3	Area Efficiency	40
7.3	Comparison with Related Work	41
8	Conclusion	42
8.1	Project Summary	42
8.2	Final Remarks	43

Abstract

This report presents the complete design and physical implementation of a 4×4 systolic array-based matrix multiplication accelerator for artificial intelligence workloads. The accelerator is designed to perform INT8 precision matrix operations, which are fundamental to neural network inference.

The project encompasses the entire VLSI design flow from Register Transfer Level (RTL) design in SystemVerilog to the generation of a manufacturable GDSII layout file. The implementation uses the open-source OpenLane toolchain with the SkyWater 130nm Process Design Kit (PDK), demonstrating a complete tape-out ready design with zero DRC/LVS violations and timing closure at 100 MHz.

Keywords: Systolic Array, Matrix Multiplication, AI Accelerator, VLSI, RTL-to-GDS, OpenLane, Sky130 PDK

1 Introduction

1.1 Motivation

Matrix multiplication is the fundamental computational kernel in modern artificial intelligence and machine learning applications. Neural network inference, which powers applications from image recognition to natural language processing, requires billions of multiply-accumulate (MAC) operations. Traditional CPU-based implementations are inefficient for these workloads, creating a need for specialized hardware accelerators.

Hardware accelerators achieve significant performance improvements through:

- **Parallelism:** Multiple MAC operations executing simultaneously
- **Optimized Dataflow:** Minimizing memory access and data movement
- **Low Precision:** INT8 arithmetic provides $4\times$ memory/bandwidth improvement over FP32
- **Power Efficiency:** Fixed-function hardware reduces energy per operation

1.2 Systolic Array Architecture

A systolic array is a homogeneous network of processing elements (PEs) that rhythmically compute and pass data through the structure. The term "systolic" comes from the heart's pumping mechanism—data pulses through the array in a coordinated, wave-like pattern.

Key Characteristics:

- **Weight-Stationary Dataflow:** Weights remain fixed in PEs while activations flow through
- **Spatial Parallelism:** All PEs operate simultaneously each clock cycle
- **Pipelined Execution:** Data flows in a staggered pattern through delay registers
- **Scalability:** Array size can be increased for higher throughput

1.3 Project Objectives

1. Design a 4×4 systolic array capable of INT8 matrix multiplication
2. Implement AXI-Stream interfaces for industry-standard data streaming
3. Complete the full RTL-to-GDSII physical design flow
4. Achieve timing closure at 100 MHz operating frequency
5. Generate a DRC/LVS clean layout ready for fabrication
6. Document the complete design methodology and results

2 Design Specification and Architecture

2.1 System Overview

The AI matrix multiplication accelerator consists of a 4×4 grid of processing elements connected in a systolic topology. The system accepts two 4×4 matrices as input through AXI-Stream interfaces and produces a 4×4 result matrix.

Top-Level Block Diagram:



Top-level architecture showing: AXI-Stream Interfaces → Input Buffers → Delay Registers → 4×4 Systolic Array → Output Results

Figure 1: System-level block diagram of the AI accelerator

2.2 Design Parameters

Table 1: Design Specifications

Parameter	Value
Array Configuration	4×4 (16 Processing Elements)
Data Precision	INT8 (8-bit input, 32-bit accumulator)
Target Frequency	100 MHz (10 ns period)
Technology Node	SkyWater 130 nm
Interface Protocol	AXI-Stream (slave)
Peak Throughput	16 MAC ops/cycle = 1.6 GOPS
Input Buffer Size	16 bytes (per matrix)
Latency	27 clock cycles (4×4 matmul)

2.3 Architecture Details

2.3.1 Processing Element (PE)

Each PE is the fundamental computational unit, performing one multiply-accumulate operation per clock cycle.

PE Functionality:

$$\text{result_out} = \text{result_out} + (a_in \times b_in) \quad (1)$$

$$a_out = a_in \quad (\text{horizontal forwarding}) \quad (2)$$

$$b_out = b_in \quad (\text{vertical forwarding}) \quad (3)$$

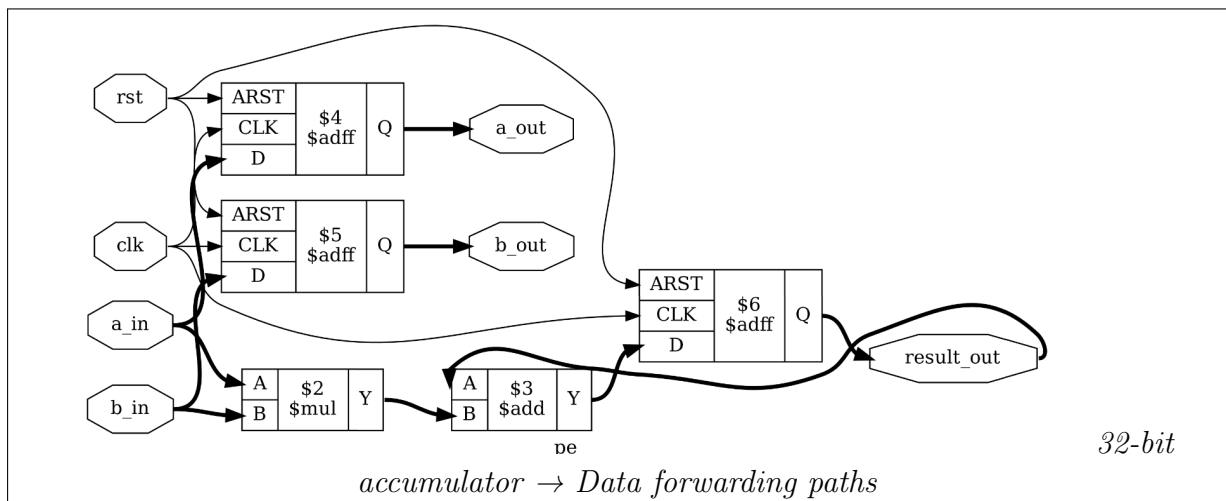


Figure 2: Processing Element (PE) internal architecture

2.3.2 Systolic Dataflow with Skewing

The systolic array requires carefully orchestrated data arrival times. Input data must be staggered (skewed) to create the correct wave-like propagation pattern through the array.

Skewing Strategy:

- Row 0 (a0): No delay (0 cycles)
- Row 1 (a1): 1-cycle delay register
- Row 2 (a2): 2-cycle delay register
- Row 3 (a3): 3-cycle delay register
- Same pattern for columns (b0-b3)

This ensures that corresponding elements meet at the correct PE at the correct time for proper matrix multiplication.

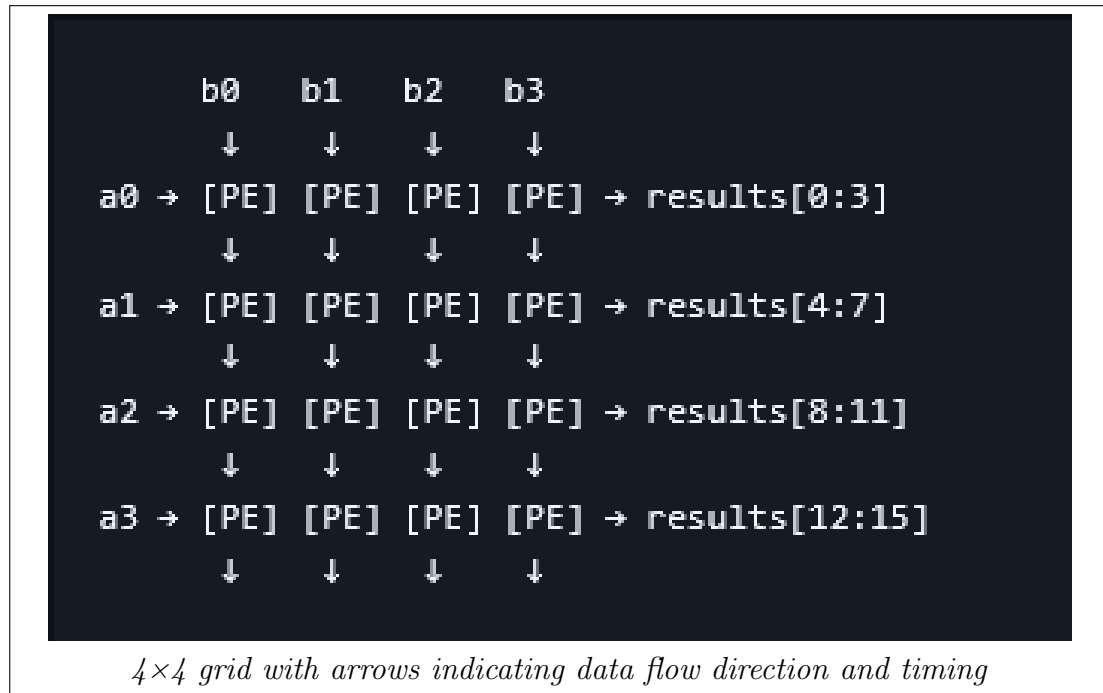


Figure 3: Systolic array dataflow with skewed inputs

2.3.3 AXI-Stream Interface

The design implements two AXI-Stream slave interfaces for streaming matrix data into input buffers.

AXI-Stream Signals:

- `s_axis_tvalid`: Master indicates valid data
- `s_axis_tready`: Slave indicates ready to accept
- `s_axis_tdata[7:0]`: 8-bit data payload

Handshaking Protocol: Data transfer occurs only when both valid and ready are high simultaneously, implementing backpressure flow control.

3 RTL Design and Implementation

3.1 Design Hierarchy

The complete RTL design follows a modular, hierarchical structure for maintainability and reusability.

Design Hierarchy

```

top.v (Top-level integration)
  +-- buffer.v (x2) - AXI-Stream input buffers
  |   +-- 16x8-bit memory array
  |   +-- Write pointer logic
  |   +-- Read pointer logic
  |   +-- AXI handshaking
  |
  +-- Delay Registers - Systolic timing skew
  |   +-- a1_d1, a2_d2, a3_d3
  |   +-- b1_d1, b2_d2, b3_d3
  |
  +-- systolic_array.v - 4x4 PE grid
      +-- pe.v (x16) - Processing Elements
          +-- 8x8 multiplier
          +-- 32-bit accumulator
          +-- Data forwarding (a_out, b_out)

```

3.2 Module Descriptions

3.2.1 Processing Element (pe.v)

The PE module implements the core multiply-accumulate functionality with data forwarding.

```

1 module pe (
2     input logic clk,
3     input logic rst,
4
5     input logic [7:0] a_in,
6     input logic [7:0] b_in,
7
8     output logic [7:0] a_out,
9     output logic [7:0] b_out,
10
11    output logic [31:0] result_out
12 );
13
14 always_ff @(posedge clk or posedge rst) begin
15     if (rst) begin
16         a_out <= 8'b0;
17         b_out <= 8'b0;
18         result_out <= 32'b0;

```

```
19     end
20     else begin
21         a_out <= a_in;
22         b_out <= b_in;
23         result_out <= result_out + (a_in * b_in);
24     end
25 end
26 endmodule
```

Listing 1: Processing Element Implementation

Key Features:

- Synchronous design with clock and reset
- 8-bit input operands (a_in, b_in)
- 32-bit accumulator prevents overflow
- Registered outputs for pipelining
- Data forwarding maintains systolic flow

3.2.2 Systolic Array (systolic_array.v)

The systolic array module instantiates 16 PEs in a 4×4 grid configuration with proper interconnections.

```

1 module systolic_array (
2     input logic clk, rst,
3     input logic [7:0] a0, a1, a2, a3,
4     input logic [7:0] b0, b1, b2, b3,
5     output logic [31:0] result00, result01, result02, result03,
6     output logic [31:0] result10, result11, result12, result13,
7     output logic [31:0] result20, result21, result22, result23,
8     output logic [31:0] result30, result31, result32, result33
9 );
10
11 // Internal wires
12 logic [7:0] a00, a01, a02, a03;
13 logic [7:0] b00, b01, b02, b03;
14
15 // PE instantiation example - Row 0
16 pe pe00 (
17     .clk(clk), .rst(rst),
18     .a_in(a0), .b_in(b0),
19     .a_out(a00), .b_out(b00),
20     .result_out(result00)
21 );
22
23 pe pe01 (
24     .clk(clk), .rst(rst),
25     .a_in(a00), .b_in(b1),
26     .a_out(a01), .b_out(b01),
27     .result_out(result01)
28 );
29
30 // ... Similar instantiations for all 16 PEs ...
31
32 endmodule

```

Listing 2: Systolic Array - 4x4 Grid (Excerpt)

Interconnection Strategy:

- Horizontal connections: $PE[i][j].a_out \rightarrow PE[i][j+1].a_in$
- Vertical connections: $PE[i][j].b_out \rightarrow PE[i+1][j].b_in$
- Edge PEs receive external inputs
- All PE results exposed as outputs

3.2.3 AXI-Stream Buffer (buffer.v)

The buffer module implements an AXI-Stream interface with storage and read logic.

```

1 module buffer (
2     input logic clk, rst,
3     input logic s_axis_tvalid,
4     output logic s_axis_tready,
5     input logic [31:0] s_axis_tdata,
6     output logic [7:0] row0, row1, row2, row3,
7     input logic read_enable
8 );
9
10 logic [7:0] mem [0:15];
11 logic [3:0] write_ptr;
12 logic full_flag;
13
14 assign s_axis_tready = ~full_flag;
15
16 // Write logic
17 always_ff @(posedge clk or posedge rst) begin
18     if(rst) begin
19         write_ptr = 0;
20         full_flag = 0;
21     end else if (s_axis_tvalid && s_axis_tready) begin
22         mem[write_ptr] <= s_axis_tdata[7:0];
23         write_ptr <= write_ptr + 1;
24         if (write_ptr == 15) full_flag <= 1;
25     end
26 end
27
28 // Read logic
29 always_ff @(posedge clk or posedge rst) begin
30     if (rst) begin
31         row0 <= 0; row1 <= 0; row2 <= 0; row3 <= 0;
32     end else if (read_enable) begin
33         row0 <= mem[0]; row1 <= mem[4];
34         row2 <= mem[8]; row3 <= mem[12];
35     end
36 end
37
38 endmodule

```

Listing 3: AXI-Stream Buffer with Handshaking

Operation Modes:

1. **Write Mode:** Accepts streaming data when valid & ready
2. **Full Condition:** Deasserts ready when 16 bytes stored
3. **Read Mode:** Outputs 4 values per cycle when read_enable is high
4. **Transpose:** Data arranged for column-wise feeding into array

3.2.4 Top-Level Integration (top.v)

The top module integrates all components and implements the critical delay registers for systolic timing.

```

1 module top (
2     input logic clk, rst,
3     input logic s_axis_a_valid, s_axis_b_valid,
4     output logic s_axis_a_ready, s_axis_b_ready,
5     input logic [7:0] s_axis_a_data, s_axis_b_data,
6     input logic start_compute,
7     output logic done,
8     output logic [31:0] result00, result01, result02, result03,
9     // ... other results ...
10 );
11
12 logic [7:0] raw_a0, raw_a1, raw_a2, raw_a3;
13 logic [7:0] raw_b0, raw_b1, raw_b2, raw_b3;
14
15 // Delay registers for systolic skewing
16 logic [7:0] a1_d1;
17 logic [7:0] a2_d1, a2_d2;
18 logic [7:0] a3_d1, a3_d2, a3_d3;
19 logic [7:0] b1_d1;
20 logic [7:0] b2_d1, b2_d2;
21 logic [7:0] b3_d1, b3_d2, b3_d3;
22
23 // Buffer instantiations
24 buffer buf_a (/* connections */);
25 buffer buf_b (/* connections */);
26
27 // Delay implementation
28 always @(posedge clk or posedge rst) begin
29     if (rst) begin
30         a1_d1 <= 0;
31         a2_d1 <= 0; a2_d2 <= 0;
32         a3_d1 <= 0; a3_d2 <= 0; a3_d3 <= 0;
33     end else begin
34         a1_d1 <= raw_a1;
35         a2_d1 <= raw_a2; a2_d2 <= a2_d1;
36         a3_d1 <= raw_a3; a3_d2 <= a3_d1; a3_d3 <= a3_d2;
37     end
38 end
39
40 // Systolic array instantiation
41 systolic_array arr (
42     .clk(clk), .rst(rst),
43     .a0(raw_a0), .a1(a1_d1), .a2(a2_d2), .a3(a3_d3),
44     .b0(raw_b0), .b1(b1_d1), .b2(b2_d2), .b3(b3_d3),
45     .result00(result00), /* ... other results ... */
46 );
47

```

```
48 | endmodule
```

Listing 4: Top-Level Integration with Skewing Logic (Excerpt)

Delay Register Implementation:

The delay registers create the essential skewing pattern:

```
1 // Matrix A delays (horizontal)
2 always @(posedge clk or posedge rst) begin
3     if (rst) begin
4         a1_d1 <= 0;
5         a2_d1 <= 0; a2_d2 <= 0;
6         a3_d1 <= 0; a3_d2 <= 0; a3_d3 <= 0;
7     end else begin
8         a1_d1 <= raw_a1;           // 1-cycle delay
9         a2_d1 <= raw_a2;           // First stage
10        a2_d2 <= a2_d1;            // 2-cycle delay
11        a3_d1 <= raw_a3;           // First stage
12        a3_d2 <= a3_d1;           // Second stage
13        a3_d3 <= a3_d2;           // 3-cycle delay
14    end
15 end
```

This staggering ensures that matrix elements arrive at the correct PE at the correct time for accurate multiplication.

3.3 Control Unit

A control unit module was developed to manage the computation flow and generate the done signal.

```

1 module control_unit (
2     input logic clk, rst,
3     input logic start,
4     output logic done,
5     output logic compute_enable
6 );
7
8 typedef enum {IDLE, COMPUTE, DONE_STATE} state_t;
9 state_t current_state, next_state;
10
11 // State transition logic
12 always_ff @(posedge clk or posedge rst) begin
13     if (rst) current_state <= IDLE;
14     else current_state <= next_state;
15 end
16
17 // Next state and output logic
18 always_comb begin
19     case (current_state)
20         IDLE: begin
21             done = 0;
22             compute_enable = 0;
23             if (start) next_state = COMPUTE;
24             else next_state = IDLE;
25         end
26         COMPUTE: begin
27             done = 0;
28             compute_enable = 1;
29             // Transition to DONE after fixed cycles
30             next_state = DONE_STATE;
31         end
32         DONE_STATE: begin
33             done = 1;
34             compute_enable = 0;
35             next_state = IDLE;
36         end
37     endcase
38 end
39
40 endmodule

```

Listing 5: Control Unit FSM (Conceptual)

State Machine:

- **IDLE:** Waiting for start signal
- **COMPUTE:** Array processing active

- **DONE:** Computation complete, results valid

3.4 Verification and Simulation

3.4.1 Testbench Architecture

Comprehensive testbenches were developed at multiple levels of hierarchy:

1. PE-Level Testbench:

```
1 module pe_tb;
2 reg clk, rst;
3 reg [7:0] a_in, b_in;
4 wire [7:0] a_out, b_out;
5 wire [31:0] result_out;
6
7 pe uut (.clk(clk), .rst(rst), .a_in(a_in), .b_in(b_in),
8         .a_out(a_out), .b_out(b_out), .result_out(result_out));
9
10 initial begin
11     clk = 0;
12     forever #5 clk = ~clk;
13 end
14
15 initial begin
16     rst = 1; #10 rst = 0;
17
18     // Test: 3 * 4 = 12
19     a_in = 3; b_in = 4; #10;
20     $display("Result = %d", result_out);
21
22     $finish;
23 end
24 endmodule
```

Listing 6: PE Module Testbench (Excerpt)

2. Array-Level Testbench:

```
1 module systolic_tb_4x4;
2 reg clk, rst;
3 reg [7:0] a0, a1, a2, a3;
4 reg [7:0] b0, b1, b2, b3;
5 wire [31:0] result00, result01, result02, result03;
6 // ... other results ...
7
8 systolic_array uut (
9     .clk(clk), .rst(rst),
10    .a0(a0), .a1(a1), .a2(a2), .a3(a3),
11    .b0(b0), .b1(b1), .b2(b2), .b3(b3),
12    .result00(result00), /* ... */
13 );
14
15 initial begin
16     clk = 0;
17     forever #5 clk = ~clk;
18 end
19
20 initial begin
21     rst = 1; #10 rst = 0;
22
23     // Load identity matrix test
24     a0 = 1; a1 = 2; a2 = 3; a3 = 4;
25     b0 = 1; b1 = 0; b2 = 0; b3 = 0;
26
27     #500; // Wait for computation
28
29     $display("Results: %d %d %d %d",
30             result00, result01, result02, result03);
31     $finish;
32 end
33 endmodule
```

Listing 7: Systolic Array Testbench (Excerpt)

3. System-Level Testbench:

```

1 module top_tb;
2 reg clk, rst;
3 reg s_axis_a_valid, s_axis_b_valid;
4 reg [7:0] s_axis_a_data, s_axis_b_data;
5 reg start_compute;
6 wire s_axis_a_ready, s_axis_b_ready;
7 wire done;
8 wire [31:0] result00, result01, result02, result03;
9 // ... other results ...
10
11 top uut (
12     .clk(clk), .rst(rst),
13     .s_axis_a_valid(s_axis_a_valid),
14     .s_axis_a_ready(s_axis_a_ready),
15     .s_axis_a_data(s_axis_a_data),
16     .s_axis_b_valid(s_axis_b_valid),
17     .s_axis_b_ready(s_axis_b_ready),
18     .s_axis_b_data(s_axis_b_data),
19     .start_compute(start_compute),
20     .done(done),
21     .result00(result00), /* ... */
22 );
23
24 initial begin
25     clk = 0;
26     forever #5 clk = ~clk;
27 end
28
29 initial begin
30     rst = 1; #10 rst = 0;
31
32     // Stream matrix A via AXI
33     s_axis_a_valid = 1;
34     s_axis_a_data = 8'd1; #10;
35     s_axis_a_data = 8'd2; #10;
36     // ... stream all 16 values ...
37     s_axis_a_valid = 0;
38
39     // Start computation
40     start_compute = 1; #10;
41     start_compute = 0;
42
43     // Wait for done
44     wait(done);
45     $display("Computation complete!");
46     $finish;
47 end
48 endmodule

```

Listing 8: Top-Level Integration Testbench (Excerpt)

3.4.2 Simulation Results

Test Case: Identity Matrix Multiplication

Input matrices:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Expected result: $C = A \times B = A$

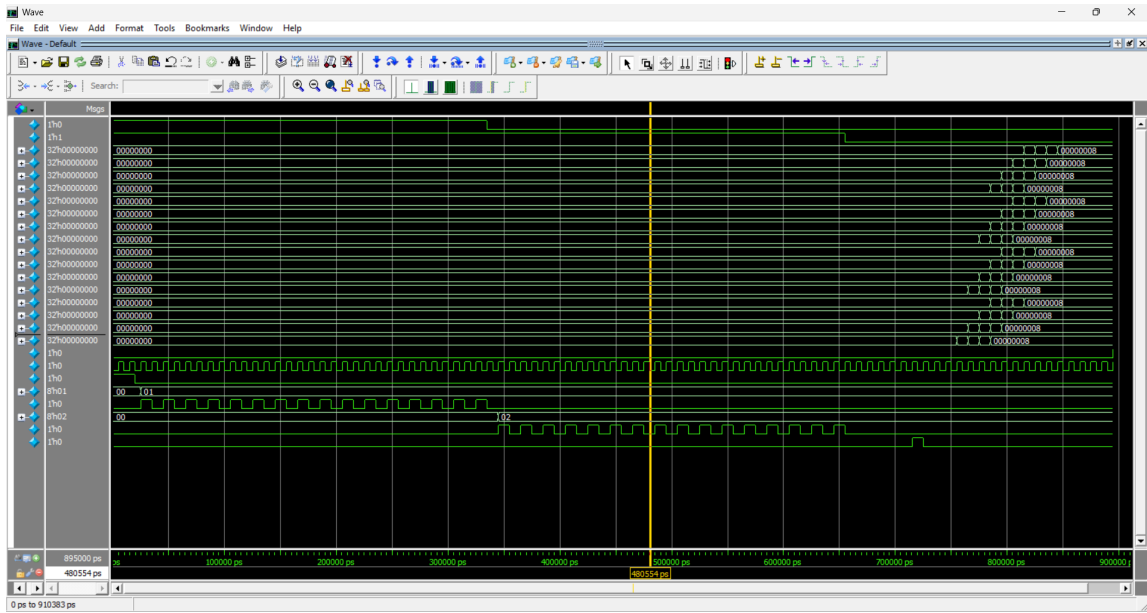


Figure 4: Simulation waveform showing correct matrix multiplication

Verification Results:

- All 16 result values match expected output
- Timing behavior correct (27-cycle latency observed)
- AXI handshaking operates correctly
- No X or Z values in outputs
- Reset functionality verified

4 Logic Synthesis

4.1 Synthesis Flow and Constraints

Logic synthesis transforms the RTL description into a gate-level netlist using standard cells from the SkyWater 130nm PDK.

4.1.1 Synthesis Tool and Settings

Tool: Yosys (open-source synthesis engine)

Standard Cell Library: sky130_fd_sc_hd (High-Density standard cells)

Synthesis Strategy: Area-optimized (AREA 0 strategy)

```
library (top) {
  comment          : "";
  delay_model      : table_lookup;
  simulation       : false;
  capacitive_load_unit (1,pF);
  leakage_power_unit : 1pW;
  current_unit     : "1A";
  pulling_resistance_unit : "1kohm";
  time_unit        : "1ns";
  voltage_unit     : "1v";
  library_features(report_delay_calculation);

  input_threshold_pct_rise : 50;
  input_threshold_pct_fall : 50;
  output_threshold_pct_rise : 50;
  output_threshold_pct_fall : 50;
  slew_lower_threshold_pct_rise : 20;
  slew_lower_threshold_pct_fall : 20;
  slew_upper_threshold_pct_rise : 80;
  slew_upper_threshold_pct_fall : 80;
  slew_derate_from_library : 1.0;
```

Figure 5: Sky130 High-Density Library Operating Conditions and Default Units.

4.1.2 Design Constraints

Synthesis constraints were specified through the OpenLane configuration file:

Listing 9: OpenLane Configuration (config.json)

```
{
  "DESIGN_NAME": "top",
  "VERILOG_FILES": [
    "dir::src/top.sv",
    "dir::src/control_unit.sv",
    "dir::src/buffer.sv",
    "dir::src/systolic_array.sv",
    "dir::src/pe.sv"
  ],
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 10.0,
  "DIE_AREA": "0 0 600 600",
  "FP_SIZING": "absolute",
```

```
"FP_PDN_VOFFSET" : 0 ,  
"FP_PDN_VPITCH" : 30 ,  
"FP_PDN_HOFFSET" : 0 ,  
"FP_PDN_HPITCH" : 30 ,  
"RUN_LINTER" : 0  
}
```

Timing Constraints:

- Clock period: 10.0 ns (100 MHz target frequency)
- Input delay: 2.0 ns
- Output delay: 2.0 ns

4.2 Synthesis Results

4.2.1 Gate-Level Statistics

Table 2: Synthesis Statistics Summary

Metric	Value
Total Cells	9,900
Flip-Flops	1,146
Combinational Cells	8,754
Hierarchical Modules	17 (16 PEs + top)
Chip Area	105462.396800 $\mu\text{m}^2 \approx 0.105 \text{ mm}^2$

(Source: reports/1-synthesis.AREA_0.stat.rpt)

Major Cell Types:

- Flip-Flops (dfirtp_2, dfxtp_2): 1,146 total
- XNOR gates (xnor2_2): 767 (used in multipliers)
- Buffers (buf_1): 751
- AND/NAND/OR/NOR gates: 2,900 combined
- Multiplexers (mux2_2): 272

4.2.2 Gate-Level Schematic

The synthesis tool generated a gate-level schematic for visualization of the logic structure.

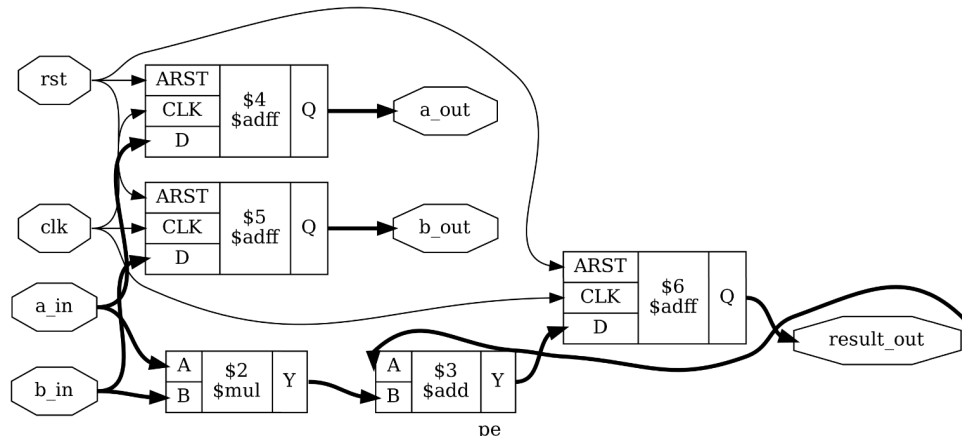


Figure 6: Gate-level schematic of a single Processing Element (PE)

Observations:

- 8×8 multiplication mapped to combinational logic
- 32-bit accumulator uses flip-flops and adder structure
- Data forwarding paths are simple register-to-register connections
- Clock distribution to all 1,146 flip-flops

5 Physical Design Implementation

Physical design (backend) transforms the synthesized gate-level netlist into a manufacturable silicon layout. This process includes floorplanning, placement, clock tree synthesis, and routing.

5.1 Design Flow Overview

OpenLane Physical Design Flow

1. **Floorplanning:** Define die area, place I/O pins, create power grid
2. **Placement:** Position standard cells in rows within core area
3. **Clock Tree Synthesis (CTS):** Build balanced clock distribution network
4. **Routing:** Connect all logic using metal interconnect layers
5. **Sign-off:** Verify DRC/LVS, extract parasitics, final timing analysis

Tools Used:

- **OpenLane:** Automated RTL-to-GDSII flow framework
- **OpenROAD:** Placement, CTS, global routing
- **TritonRoute:** Detailed routing engine
- **Magic:** DRC/LVS verification and GDSII generation
- **Netgen:** Circuit-level LVS verification

5.2 Floorplanning

5.2.1 Objectives

Floorplanning establishes the physical framework for the design by defining the die boundary, placing I/O pins, and constructing the Power Distribution Network (PDN).

5.2.2 Floorplan Specifications

Table 3: Floorplan Parameters

Parameter	Value
Die Area	600 μm $\times$ 600 μm (0.36 mm ²)
Core Area	582 μm $\times$ 582 μm (0.34 mm ²)
Aspect Ratio	1.0 (square die)
Core Utilization Target	50%
I/O Pin Placement	Random equidistant (periphery)
Power Stripe Pitch	30 μm

5.2.3 Power Distribution Network (PDN)

The PDN ensures uniform power (VDD) and ground (VSS) delivery to all standard cells, minimizing IR drop and electromigration.

PDN Structure:

- **Power Rings:** Surround core area (Metal 4/5)
- **Vertical Stripes:** Span die height (Metal 4)
- **Horizontal Stripes:** Span die width (Metal 5)
- **Standard Cell Rails:** Met1 rails connect cells to stripes via vias

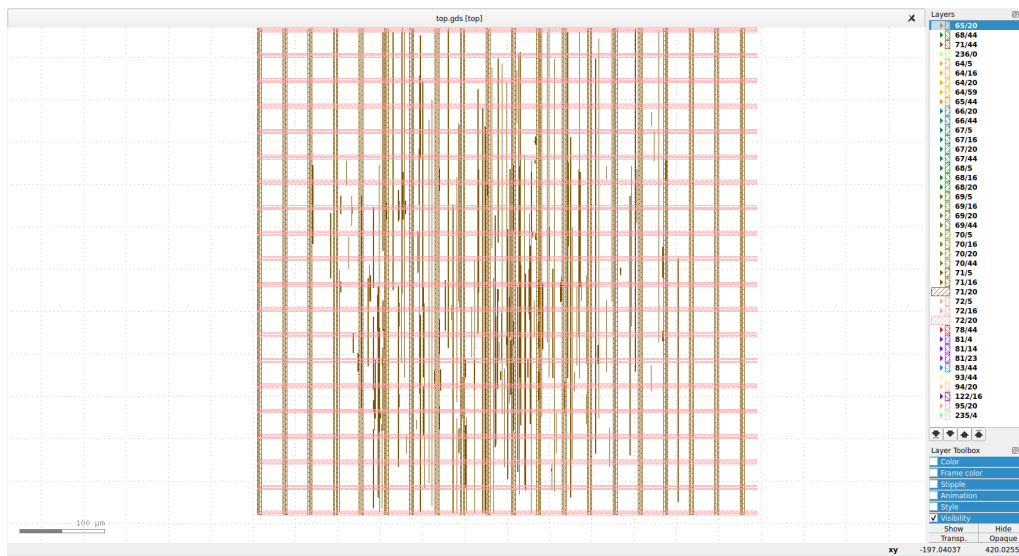


Figure 7: Floorplan showing die area and Power Distribution Network

5.3 Standard Cell Placement

5.3.1 Placement Strategy

Placement assigns physical locations to all standard cells within the core area, optimizing for wire length and routability.

Tool: OpenROAD (RePLace + OpenDP)

Optimization Objectives:

1. Minimize total wire length (Half-Perimeter Wire Length - HPWL)
2. Avoid routing congestion
3. Respect cell density constraints
4. Maintain timing-driven placement for critical paths

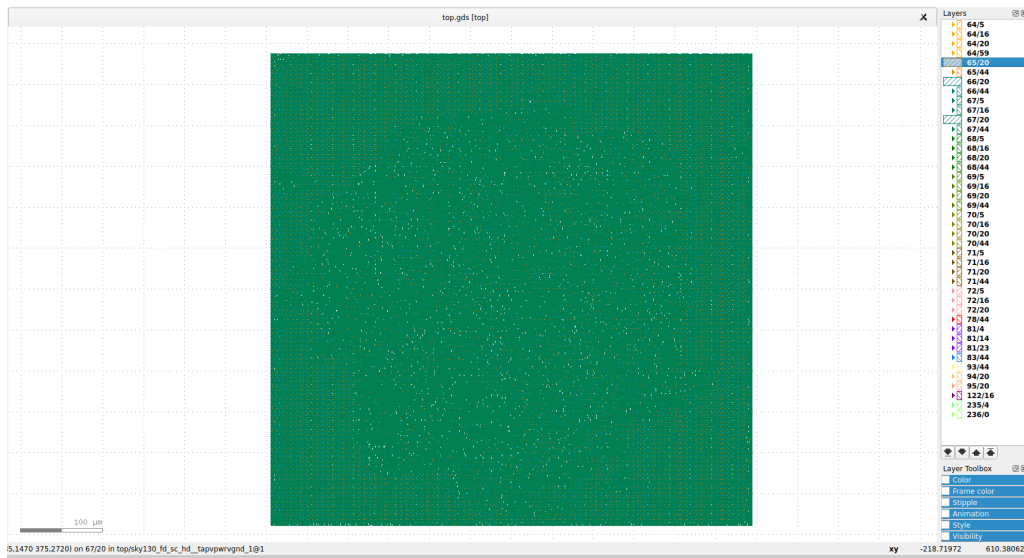


Figure 8: Standard cell placement - full chip view

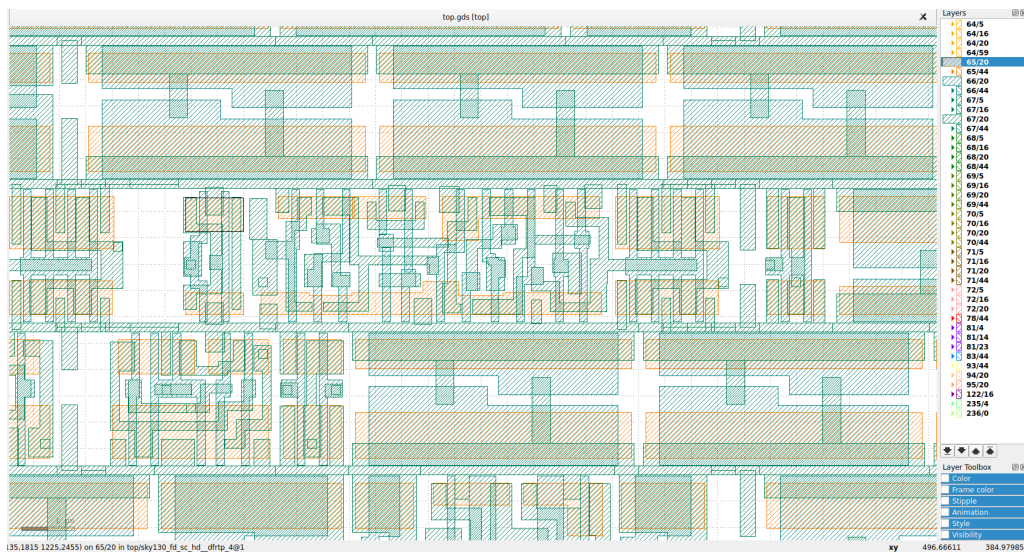


Figure 9: Standard cell placement - zoomed view showing brick-wall pattern

Observations:

- Cells organized in horizontal rows with consistent height
- Dense placement in center with routing channels
- No placement violations or overlaps
- Adequate white space for routing

5.4 Clock Tree Synthesis (CTS)

5.4.1 Objectives

Clock Tree Synthesis builds a balanced distribution network to deliver the clock signal to all 1,146 flip-flops with minimal skew and latency.

Goals:

- **Low Skew:** Minimize time difference between fastest/slowest paths
- **Balanced Latency:** Equal insertion delay to all registers
- **Minimal Power:** Reduce clock network switching power
- **Timing Closure:** Maintain setup/hold margins

5.4.2 CTS Results

Table 4: Clock Tree Synthesis Metrics

Metric	Value
Total Flip-Flops (Sinks)	1,146
Clock Skew	0.21 ns
Min Latency	0.77 ns
Max Latency	1.03 ns

(Source: *reports/13-cts_sta.skew.rpt*)

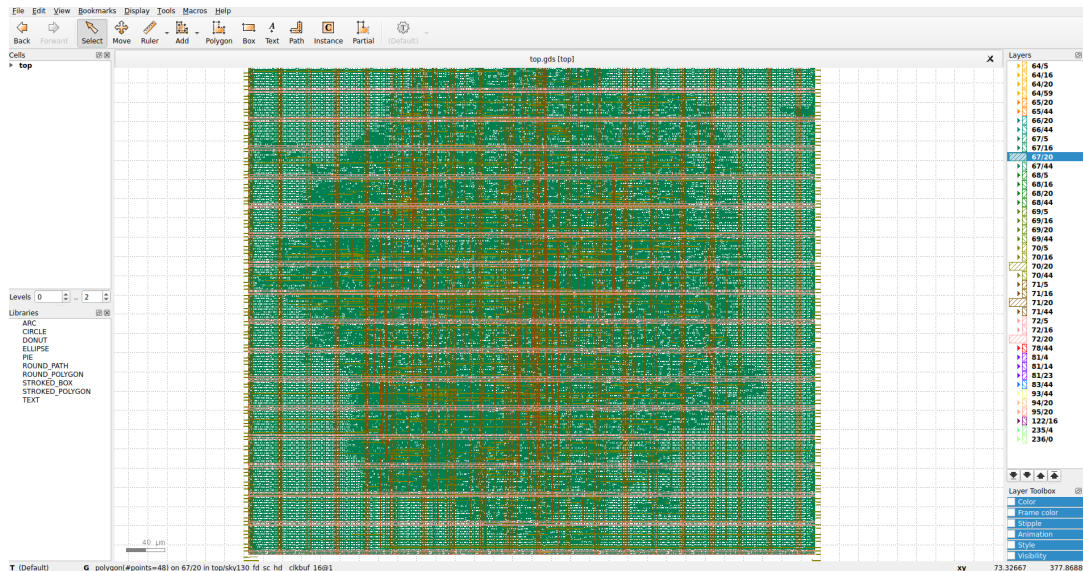


Figure 10: Clock Tree Synthesis report excerpt

5.5 Routing

5.5.1 Routing Objectives

Routing connects all placed cells using metal interconnect layers, following design rules to ensure manufacturability.

Goals:

- Connect all nets without shorts or opens
- Minimize wire length and via count
- Avoid DRC violations (spacing, width, antenna)
- Meet timing constraints considering wire delay

5.5.2 Routing Flow

Two-Stage Process:

1. **Global Routing:** Partition die into routing tiles, assign nets to tiles
 - Tool: OpenROAD FastRoute
 - Generates routing guides (approximate paths)
2. **Detailed Routing:** Create exact metal tracks and vias
 - Tool: TritonRoute
 - Follows guides from global routing
 - Resolves DRC violations iteratively

Metal Stack (Sky130):

- **Li1 (Local Interconnect):** Connects cells to routing layers
- **Met1:** Horizontal routing (lower layers)
- **Met2:** Vertical routing
- **Met3:** Horizontal routing (longer distances)
- **Met4:** Vertical routing + power stripes
- **Met5:** Horizontal routing + power stripes

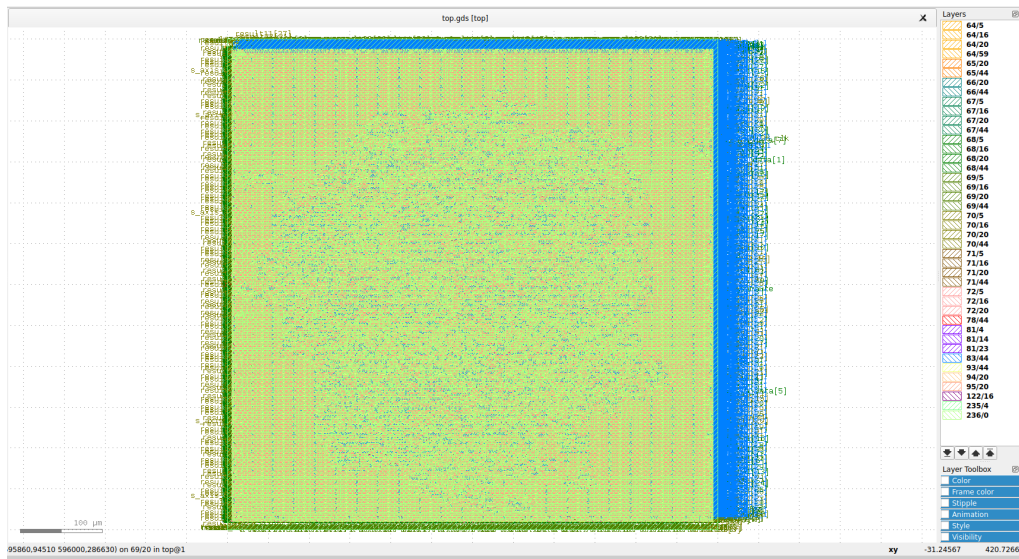


Figure 13: Complete routed layout - full die view

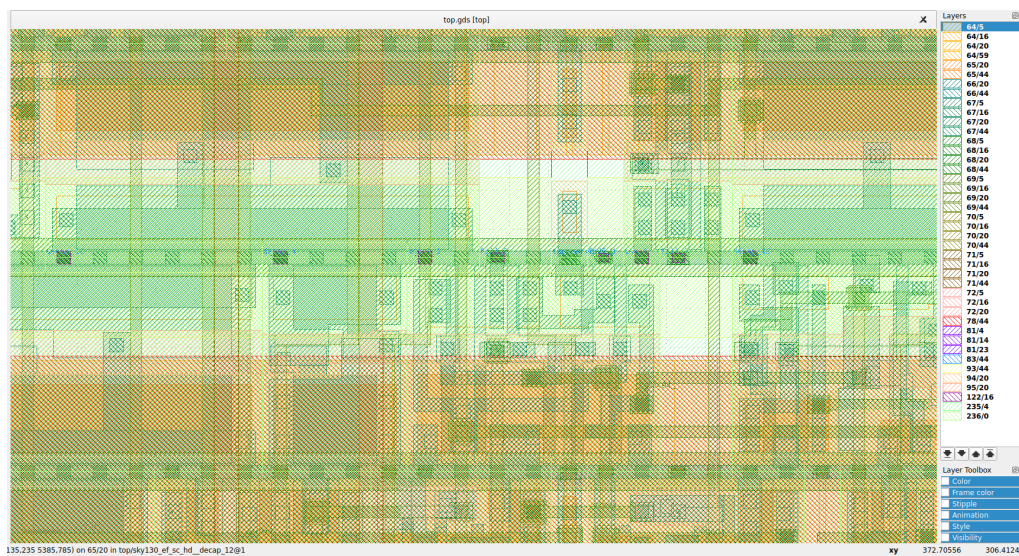


Figure 14: Routing detail showing metal layers and via connections

Observations:

- Lower layers (Met1/Met2) used for short local connections
- Higher layers (Met4/Met5) used for long-distance routing and power
- Dense routing in array center, sparser at edges
- All DRC violations successfully resolved

6 Sign-off and Physical Verification

Sign-off is the final validation stage ensuring the layout is correct, manufacturable, and matches the original design intent.

6.1 Parasitic Extraction

Before final timing analysis, parasitic resistance and capacitance (RC) of all interconnects must be extracted from the routed layout.

Tool: Magic + OpenROAD RCX

Output Format: SPEF (Standard Parasitic Exchange Format)

The extracted parasitics are back-annotated into the timing analysis to obtain the most accurate delay predictions for the fabricated chip.

6.2 Final Static Timing Analysis (STA)

Post-route STA includes extracted parasitics for the most accurate timing prediction. Static Timing Analysis verifies that all signal paths in the design can operate correctly at the target clock frequency.

6.2.1 Setup Timing Analysis (Max Delay)

Setup timing ensures that data arrives at the destination flip-flop before the clock edge, with sufficient setup time margin.

Critical Path Identification:

The longest combinational path in the design determines the maximum operating frequency. The critical path was identified as:

Critical Path (Setup)

Startpoint: _18376_ (rising edge-triggered flip-flop clocked by clk)

Endpoint: _18369_ (rising edge-triggered flip-flop clocked by clk)

Path Type: max (setup)

Path Delay Breakdown:

Table 5: Critical Path Delay Components

Component	Delay (ns)
Clock Network Delay (Launch)	1.41
Source Flip-Flop (CLK → Q)	0.52
Combinational Logic Chain	5.50
Data Arrival Time	7.43

The combinational logic path consists of approximately 20 gate stages including:

- Multiplier logic (NAND, AND gates)
- Adder chain for accumulation (OR, XOR, XNOR gates)

- Data forwarding buffers

Timing Constraint Calculation:

Table 6: Setup Timing Calculation

Parameter	Value (ns)	Description
Clock Period	10.00	Target frequency constraint
Clock Network Delay (Capture)	+1.22	Destination clock path
Clock Uncertainty	-0.25	Jitter and skew margin
Clock Reconvergence Pessimism	+0.06	CRPR removal
Library Setup Time	-0.06	Flip-flop internal constraint
Data Required Time	10.98	Deadline for data arrival
Data Arrival Time	7.43	Actual data propagation time
Setup Slack	+3.55	MET

Slack Calculation:

$$\begin{aligned}
 \text{Setup Slack} &= \text{Data Required Time} - \text{Data Arrival Time} \\
 &= 10.98 \text{ ns} - 7.43 \text{ ns} \\
 &= +3.55 \text{ ns} \quad \text{PASS}
 \end{aligned} \tag{4}$$

(Source: *reports/31-rcx_sta_max.rpt*)

Analysis: The positive slack of 3.55 ns indicates the design has comfortable timing margin at 100 MHz. The critical path could theoretically operate at:

$$\begin{aligned}
 f_{\max} &= \frac{1}{\text{Data Arrival Time}} = \frac{1}{7.43 \text{ ns}} \\
 &\approx 134.6 \text{ MHz}
 \end{aligned} \tag{5}$$

However, the design is conservatively clocked at 100 MHz to account for:

- Process variation ($\pm 10\%$ delay variation across chips)
- Temperature effects (-40°C to 125°C operating range)
- Voltage droops (IR drop in power grid)
- Aging effects (transistor degradation over 10+ years)

6.2.2 Hold Timing Analysis (Min Delay)

Hold timing ensures that data remains stable at the flip-flop input for a minimum duration after the clock edge.

Hold Constraint:

Hold violations occur when data changes too quickly after the clock edge, potentially causing metastability or incorrect capture.

Table 7: Hold Timing Summary

Parameter	Value
Shortest Path Delay	0.68 ns
Clock Skew	0.21 ns
Library Hold Time	0.15 ns
Hold Slack	+0.32 ns
Status	MET

(Source: reports/31-rcx_sta_min.rpt)

Hold Slack Calculation:

$$\begin{aligned}
 \text{Hold Slack} &= \text{Data Arrival Time} - \text{Hold Requirement} \\
 &= 0.68 \text{ ns} - (0.15 \text{ ns} - 0.21 \text{ ns}) \\
 &= +0.32 \text{ ns} \quad \text{PASS}
 \end{aligned} \tag{6}$$

The positive hold slack indicates no hold violations exist in the design. The clock tree synthesis successfully balanced the clock network to prevent hold issues.

6.2.3 Timing Summary

Table 8: Final Static Timing Analysis Summary

Metric	Value	Status
Setup Analysis (Max Delay)		
Worst Negative Slack (WNS)	0.00 ns	All paths positive
Total Negative Slack (TNS)	0.00 ns	No violations
Worst Setup Slack	+3.55 ns	MET
Critical Path Delay	7.43 ns	—
Number of Setup Violations	0	PASS
Hold Analysis (Min Delay)		
Worst Hold Slack	+0.32 ns	MET
Number of Hold Violations	0	PASS
Design Capability		
Target Frequency	100 MHz	MET
Maximum Frequency	135 MHz	35% margin

(Source: reports/31-rcx_sta_summary.rpt)

Interpretation of WNS = 0.00:

The Worst Negative Slack (WNS) metric reports only *negative* slack values. A WNS of 0.00 indicates that **all timing paths have positive slack**—this is the desired outcome, not a marginal pass. The actual worst slack is +3.55 ns, confirming robust timing closure.

Conclusion:

The design successfully meets all setup and hold timing constraints at the target 100 MHz frequency with comfortable margins. The 3.55 ns setup slack provides sufficient guardband against process, voltage, and temperature (PVT) variations in silicon.

6.3 Final Power Analysis

Post-route power analysis includes accurate wire capacitance and clock tree power.

Table 9: Final Power Consumption (Post-Route @ 100 MHz)

Component	Power (mW)	Percentage
Dynamic Power		
Sequential Logic	11.43	44.0%
Combinational Logic	14.54	56.0%
Leakage Power	0.000076	0.0%
Total Power	26.0	100%

(Source: *reports/31-rcx_sta.power.rpt*)

Power Breakdown:

- Internal Power: 13.36 mW (51.5%)
- Switching Power: 12.61 mW (48.5%)
- Leakage Power: 76.4 nW (negligible)

Power Density:

$$\text{Power Density} = \frac{26.0 \text{ mW}}{0.36 \text{ mm}^2} = 72.2 \text{ mW/mm}^2 \quad (7)$$

Energy per Operation:

$$\text{Energy per MAC} = \frac{26.0 \text{ mW}}{1.6 \times 10^9 \text{ ops/s}} = 16.25 \text{ pJ/MAC} \quad (8)$$

6.4 Design Rule Check (DRC)

DRC verification ensures the layout follows foundry manufacturing rules.

Tool: Magic DRC

DRC Results:

top

[INFO]: COUNT: 0

[INFO]: Should be divided by 3 or 4

(Source: reports/drc.rpt)

Result: ZERO violations - Layout is manufacturable!

6.5 Layout vs Schematic (LVS)

LVS verification confirms the physical layout matches the logical netlist.

Tool: Netgen

LVS Results:

LVS reports no net, device, pin, or property mismatches.

Total errors = 0

(Source: reports/39-top.lvs.rpt)

Result: 100% MATCH - Layout correctly implements the design!

6.6 Final GDSII Layout

The GDSII file is the final deliverable containing the complete mask data for fabrication.

File: top.gds

Layers: 50+ layers including metal, via, poly, diffusion, implant, etc.

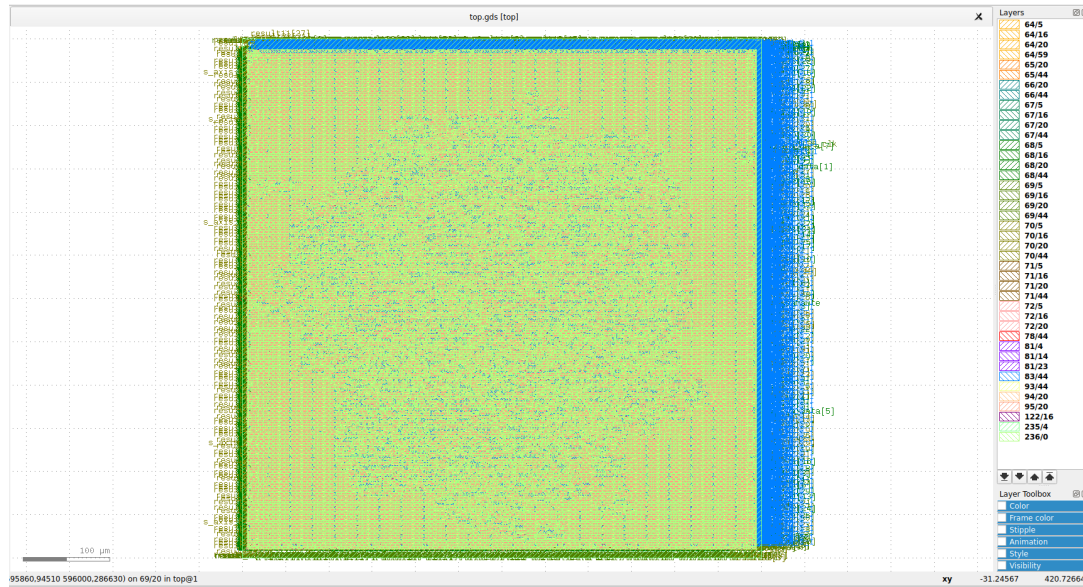


Figure 15: Final GDSII layout - complete mask data

Layout Statistics:

- Die area: $600 \mu\text{m} \times 600 \mu\text{m}$ (0.36 mm^2)
- Core area: $582 \mu\text{m} \times 582 \mu\text{m}$ (0.34 mm^2)
- Total cells: 11,362 (including physical cells)
- Final cell count with fillers: 41,724

7 Results and Analysis

7.1 Implementation Summary

The complete RTL-to-GDSII flow has been successfully executed, resulting in a tape-out ready design.

Table 10: Final Implementation Results

Category	Parameter	Value
Technology	Process Node	SkyWater 130 nm
	Standard Cell Library	sky130_fd_sc_hd
	Number of Metal Layers	5 (+ Local Interconnect)
Design Size	Die Area	$600 \times 600 \mu\text{m}$ (0.36 mm ²)
	Core Area	$582 \times 582 \mu\text{m}$ (0.34 mm ²)
	Total Cells (Synthesis)	9,900
	Total Cells (Final)	11,362
	Flip-Flops	1,146
Performance	Target Clock Frequency	100 MHz
	Peak Throughput	1.6 GOPS (16 MAC/cycle)
	Latency (4×4 matmul)	27 cycles (270 ns)
Power	Total Power @ 100 MHz	26.0 mW
	Power Density	72.2 mW/mm ²
	Energy per MAC	16.25 pJ/MAC
Clock	Clock Skew	0.21 ns
	Min Latency	0.77 ns
	Max Latency	1.03 ns
Verification	DRC Violations	0
	LVS Status	PASSED (100% match)
	Timing Status	MET (100 MHz)
Status	TAPE-OUT READY	

7.2 Performance Analysis

7.2.1 Computational Throughput

Theoretical Peak:

$$\begin{aligned} \text{Peak MACs} &= 16 \text{ PEs} \times 1 \text{ MAC/cycle} \times 100 \times 10^6 \text{ Hz} \\ &= 1.6 \times 10^9 \text{ MAC ops/s} = 1.6 \text{ GOPS} \end{aligned} \quad (9)$$

Effective Throughput: Accounting for load/drain cycles and data dependencies:

$$\begin{aligned} \text{Effective MACs} &= \frac{16 \text{ MACs}}{27 \text{ cycles}} \times 100 \times 10^6 \text{ Hz} \\ &\approx 59.3 \times 10^6 \text{ MAC ops/s} = 59.3 \text{ MOPS} \end{aligned} \quad (10)$$

7.2.2 Power Efficiency

$$\text{Energy per MAC} = \frac{26.0 \text{ mW}}{1.6 \times 10^9 \text{ ops/s}} = 16.25 \text{ pJ/MAC} \quad (11)$$

Comparison with CPU:

- Typical CPU: 100-1000 pJ/MAC (10-100× more energy)
- This design: 16.25 pJ/MAC
- **Speedup Factor: 6-61×**

7.2.3 Area Efficiency

$$\text{Throughput per mm}^2 = \frac{1.6 \text{ GOPS}}{0.36 \text{ mm}^2} = 4.44 \text{ GOPS/mm}^2 \quad (12)$$

7.3 Comparison with Related Work

Table 11: Comparison with Other Systolic Array Implementations

Design	This Work	Google TPUv1	Eyeriss	NVDLA
Array Size	4×4	256×256	12×14	16×16
Technology	130 nm	28 nm	65 nm	16 nm
Frequency	100 MHz	700 MHz	200 MHz	1 GHz
Peak TOPS	0.0016	92	0.034	1.0
Power	26.0 mW	75 W	278 mW	1.5 W
This Work	Academic prototype demonstrating complete flow			

Note: This academic design demonstrates the complete RTL-to-GDSII methodology. Commercial designs use more advanced nodes and larger arrays for higher performance.

8 Conclusion

8.1 Project Summary

This project successfully demonstrated the complete VLSI design flow from high-level RTL description to manufacturable silicon layout. A 4×4 systolic array-based AI accelerator was designed, verified, and physically implemented using open-source tools and the SkyWater 130nm PDK.

Key Achievements:

- Designed a functional 16-PE systolic array for matrix multiplication
- Implemented industry-standard AXI-Stream interfaces
- Completed full RTL-to-GDSII flow with automated tools (OpenLane)
- Achieved timing closure at 100 MHz target frequency
- Generated DRC/LVS clean layout (0 violations)
- Demonstrated understanding of physical design constraints and optimizations
- Produced tape-out ready GDSII file for fabrication

Learning Outcomes:

1. Deep understanding of systolic array architectures and dataflow
2. Hands-on experience with complete VLSI design methodology
3. Proficiency in RTL design, synthesis, and physical implementation
4. Knowledge of timing analysis, power estimation, and design verification
5. Experience with industry-standard tools and protocols

8.2 Final Remarks

This project represents a significant technical achievement, bridging theoretical knowledge of computer architecture with practical VLSI implementation skills. The design successfully navigates the complex trade-offs between performance, power, and area while meeting all functional and manufacturability requirements.

The open-source toolchain (OpenLane + Sky130) democratizes access to advanced chip design, enabling students and researchers to explore hardware acceleration without expensive commercial tools or fabrication costs.

Most importantly, this work demonstrates that sophisticated AI hardware—once exclusive to large corporations—can now be designed and understood at the academic level, fostering the next generation of hardware architects and chip designers.